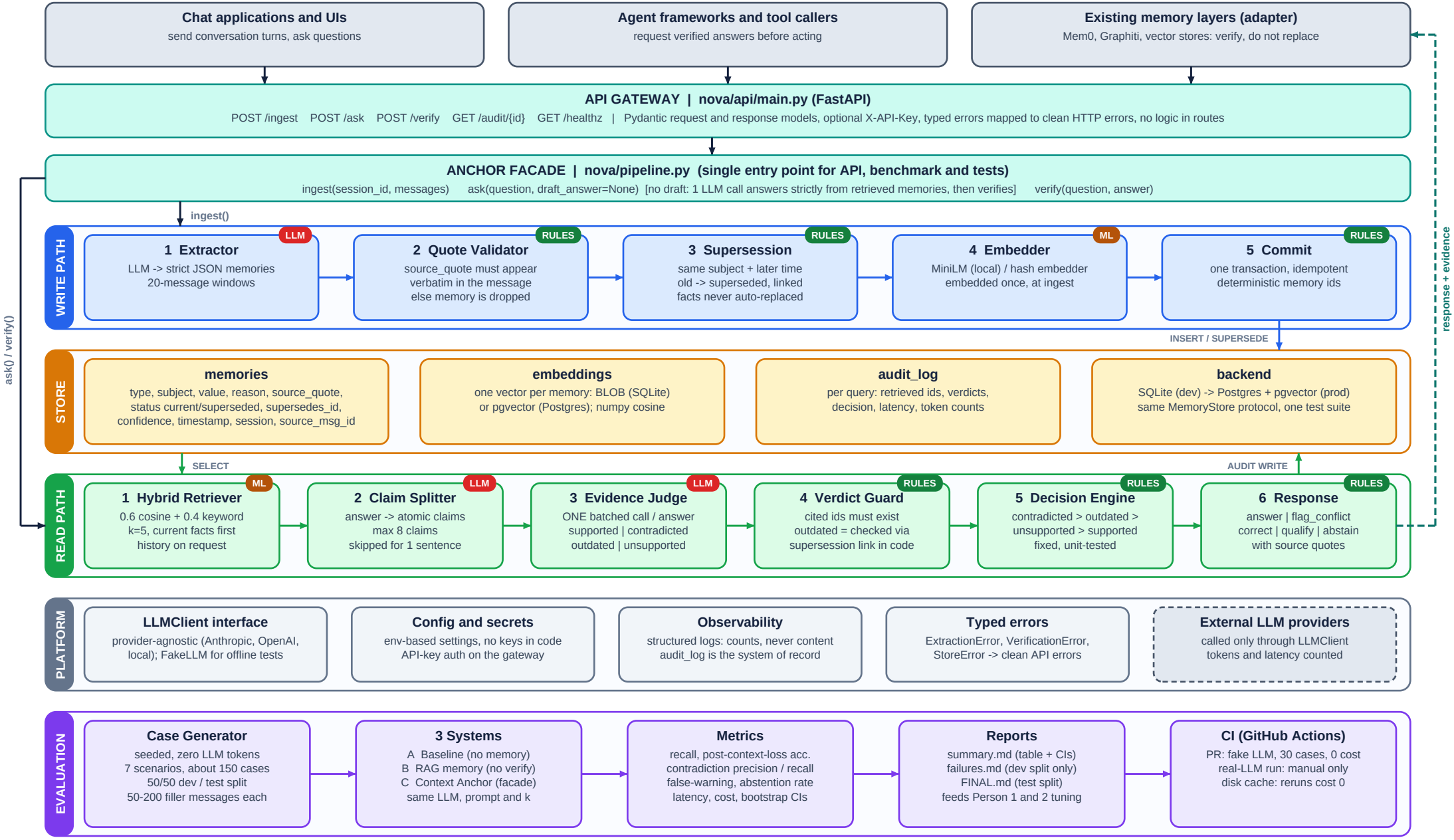
CONTEXT ANCHOR | SYSTEM ARCHITECTURE

External source-linked memory + evidence-checked answers. Model-independent, auditable, benchmarked.

v1.0 | 24 Oct 2026

github.com/Devanshi-Tech3158/nova



LEGEND LLM calls LLMClient, uses API tokens | RULES deterministic code, zero tokens | ML local model, zero API tokens | solid arrow = data flow | dashed arrow = response / feedback

Design priorities: (1) minimum tokens: batched judge, fast paths, caching (2) verifiable: source quotes, version chain, audit (3) swappable: protocols for store, LLM, embedder, retriever

Absence of evidence is never reported as false. Final actions are decided by deterministic code, not by an LLM.

1. Component specification

Every component sits behind a typed protocol in nova/core, so any one can be swapped (SQLite to Postgres, local embeddings to an API, one LLM provider to another) without touching the rest. Core schemas and protocols are frozen after the contract commit.

Module	Responsibility	Mode	Key design decisions	Owner / phase
api/main.py	FastAPI gateway: /ingest, /ask, /verify, /audit/{id}, /healthz	RULES	No business logic in routes; optional X-API-Key from env; typed exceptions mapped to clean errors with no stack traces	P1, phase 5
pipeline.py (Anchor)	Single entry point used by API, benchmark and tests	MIXED	ingest() by P1, ask() and verify() by P2; idempotent re-ingest; counts-only structured logging	P1 + P2, ph. 4-5
extraction/extractor.py	Messages to validated JSON memories (fact, decision, correction, preference, commitment)	LLM	Windows of 20 messages bound token use; Pydantic validation; one repair retry, then ExtractionError	P1, phase 2
extraction/quote_validator.py	Drops any memory whose source_quote is not in the cited message	RULES	Whitespace-normalized, case-insensitive substring test; deterministic id = sha1(session, msg, subject, value)	P1, phase 2
temporal/supersession.py	Keeps current versus outdated truth	RULES	Same subject + later timestamp supersedes decisions, corrections, preferences; handles out-of-order ingest; facts and commitments never auto-replaced; idempotent	P1, phase 3
store/sqlite_store.py, pg_store.py	Persistent MemoryStore with source metadata	RULES	WAL mode, parameterized SQL only, atomic supersede, schema_version migrations; one contract test suite runs on both backends	P1, ph. 1 and 6
retrieval/hybrid.py	Finds relevant memories across sessions	ML	0.6 cosine + 0.4 keyword/subject overlap; current facts by default, history on request; numpy cosine (no vector DB needed at this scale); deterministic ties	P1, phase 3b
verification/claim_splitter.py	Answer to atomic, checkable claims	LLM	Maximum 8 claims; single-sentence fast path skips the LLM; strict JSON with one repair retry	P2, phase 1
verification/judge.py	Labels each claim against retrieved evidence	LLM	One batched call per answer; four labels; code checks that cited ids exist; missing claims become unsupported	P2, phase 2
Verdict guard (in judge.py)	Stops the judge from inventing evidence	RULES	Hallucinated ids downgrade to unsupported; outdated is verified through the supersession link, not trusted from the LLM	P2, phase 2
verification/decision.py	Turns verdicts into an action	RULES	Priority contradicted > outdated > unsupported > supported; deterministic response templates; full table-driven tests	P2, phase 3
audit/logger.py	One audit record per query	RULES	Stores retrieved ids, verdicts, action, latency, tokens; text truncated to 2000 chars; never stores keys	P2, phase 4
core/ (schemas, protocols, llm, config)	The contract everything codes against	RULES	Pydantic v2 models; LLMClient + FakeLLM; env-based config; changes only with human approval	P1, phase 0
benchmark/	Proves the system reduces errors	MIXED	Seeded zero-LLM generator, three systems, deterministic metrics, bootstrap CIs, disk-cached LLM calls (see section 4)	P3, phases 1-6

2. Data model, decision logic and runtime flows

Memory field	Type	Purpose
id	str	Deterministic hash of session, message, subject, value (idempotent ingest)
type	enum	fact, decision, correction, preference, commitment
subject	str	Normalized snake_case key (project_database); the supersession key
value / reason	str / str?	The stored statement or choice, and why it was chosen over alternatives
source_msg_id	str	Original message, for traceability
source_quote	str	Verbatim text, validated as a substring of that message
session_id, timestamp	str, datetime	Cross-session recovery and temporal ordering
status	enum	current or superseded; history is never deleted
supersedes_id	str?	Version chain: the memory this one replaced
confidence	float	Extractor confidence, carried into verdicts and audit
embedding	blob	Stored beside the memory in its own column

Evidence state	Verdict mix	Action	User receives
Supported	all claims supported	answer	Answer with cited source quotes
Conflict	any contradicted	flag_conflict	Both sources shown; no silent override
Outdated	any outdated	correct	Current value, noting it replaced the old one
Partial	some supported, some unsupported	qualify	Supported part stated; rest marked unverified
Insufficient	nothing supported, or no memories	abstain	Says evidence is missing; may ask to clarify

Operation	LLM calls	Notes
Ingest 20 messages	1	Extractor only. Validate, supersede, embed use none. Plus 1 only on a bad-JSON repair retry
Verify one answer	up to 2	Splitter (0 for a one-sentence answer) + judge (0 if no memories). Decision is code
Ask without a draft	+1	Answers strictly from retrieved memories, then verified as above
Re-ingest same messages	target 0	Design target: skip already-ingested message ids before extraction
Benchmark rerun	0	Responses cached on disk by prompt hash; PR checks use a fake LLM

Ingest sequence	Verify sequence
1 Client posts messages to /ingest 2 Facade batches them in windows of 20 3 Extractor returns JSON memories 4 Validator drops any memory lacking a verbatim quote 5 Supersession links reversals and corrections 6 Embedder vectorizes new memories 7 Store commits in one transaction; IngestReport returns (added, superseded, dropped)	1 Client sends question + draft to /verify (or /ask with no draft) 2 Retriever returns top 5 current memories plus history 3 Splitter produces atomic claims 4 Judge labels all claims in one call 5 Guard downgrades invalid ids, checks outdated in code 6 Decision engine picks the action 7 Audit record written; response returns with evidence

3. System guarantees, failure modes and mitigations

System invariant	Enforced by
1 Every stored memory carries a verbatim source quote	Quote validator rejects it otherwise (unit-tested)
2 History is never deleted, only superseded	store.supersede is atomic; the protocol has no delete
3 No evidence means unsupported, never false	Judge label rules plus code-side downgrade
4 The final action is decided by code, not by an LLM	decision.py with table-driven tests
5 LLM output is never trusted unvalidated	Pydantic schemas, id existence checks, outdated verified via supersession link
6 Same interfaces in test and production	LLMClient and FakeLLM, HashEmbedder, backend contract tests
7 Every verification is auditable	audit_log record per query: ids, verdicts, action, latency, tokens

Failure modes

Failure	Where	Mitigation	Status
Extractor invents a fact	extraction	Quote validator drops any memory whose quote is not in the source message	Specified (P1 ph. 2)
Quote is real but meaning is misread	extraction	Confidence score, benchmark scenarios, human review through the audit log	Residual risk
Reversal missed because subject names differ	supersession	Normalized subjects; decision_reversed benchmark cases; v2: embedding-based subject matching	Residual, measured
Judge cites a memory that does not exist	judge	Id check; label downgrades to unsupported	Specified (P2 ph. 2)
Judge wrongly says outdated	judge	Verified in code through the supersession link	Specified (P2 ph. 2)
Valid answers get flagged	decision	false_warning_rate reported beside contradiction recall; valid_answer cases	Measured by benchmark
Right memory is not retrieved	retrieval	Hybrid scoring and retrieval-accuracy metric; thin evidence leads to abstain, not a guess	Measured by benchmark
Provider outage or malformed JSON	LLM layer	One repair retry, typed errors, clean API error; FakeLLM keeps tests offline	Specified
Wrong or poisoned memory is stored	store	Source links, version chain, confidence and audit make it visible; cannot be fully eliminated	Residual risk
Sensitive data retained	store	Planned: identifier denylist at extraction, per-tenant scoping, retention and deletion policy	Planned, not built
Benchmark overfitting	evaluation	Dev / test split, test run once, bootstrap confidence intervals	Specified (P3 ph. 5-6)

Security and privacy posture

API key read from environment, never committed. Logs record counts only, never message content. Data stays in the operator's own database (self-hosted). Planned for v2: per-tenant isolation, retention and deletion API, encryption at rest through the database. Status labels describe the plan: the repository is still empty, so nothing here is implemented yet. Do not describe planned items as implemented in any submission.

4. Evaluation architecture

Capability	A Baseline	B RAG memory	C Context Anchor
Persistent memory beyond the window	No	Yes (raw chunks)	Yes (structured)
Source quote per fact	No	Chunk only	Verbatim, validated
Current versus outdated facts	No	No	Yes (version chain)
Contradiction detection	No	No	Yes
Abstains on missing evidence	Prompt only	Prompt only	Evidence-based
Same LLM, prompt, k	Yes	Yes	Yes

Metric	Definition
Recall	Correct on buried_fact, paraphrased, cross_session
Accuracy after context loss	Correct where context_dropped is true
Contradiction precision / recall	flag or correct on contradicting_draft versus all other cases
Unsupported-claim rate	unanswerable cases that were not abstained
Correct abstention	unanswerable cases that were abstained
Retrieval accuracy	Expected source message among retrieved ids (B and C)
False-warning rate	valid_answer cases not answered
Latency, tokens, cost	p50 and p95 per case; price table in config

Benchmark scenarios (seeded generator, about 150 cases, 50/50 dev / test split)

Scenario	Planted situation	Expected action	What it measures
buried_fact	A fact among 50-200 filler messages, asked after the context is dropped	answer	Recall after context loss
decision_reversed	A decision followed by a correction; also asks for the old value in some cases	answer (new value)	Temporal memory and version history
unanswerable	Topic never mentioned in any session	abstain	Unsupported-claim rate, correct abstention
contradicting_draft	Draft answer states the old or wrong value	flag_conflict or correct	Contradiction precision and recall
paraphrased	Question shares at most one content word with the planted message	answer	Semantic retrieval accuracy
cross_session	Fact in session 1, question in session 3	answer	Cross-session recovery
valid_answer	Correct, supported draft answer	answer	False-warning rate

Protocol rules

- Tune on the dev split only. Run the held-out test split once with the real LLM and report dev and test side by side.
- Grading is deterministic (normalized substring and action match), so no LLM-judge error enters the scores. Every metric carries a bootstrap 95% CI (1000 resamples, fixed seed); zero denominators report n/a.
- Scope note: synthetic cases show the pipeline handles controlled failures. Person 4 adds roughly 100 hand-labeled claim/evidence pairs (judge accuracy) and real-conversation checks to test generalization.

5. Repository, deployment and team plan

```
nova/
  core/          schemas.py protocols.py llm.py config.py
  store/         sqlite_store.py pg_store.py
  extraction/    extractor.py quote_validator.py prompts.py
  temporal/      supersession.py
  retrieval/     embedder.py hybrid.py
  verification/  claim_splitter.py judge.py decision.py
                 prompts.py service.py
  audit/         logger.py
  api/           main.py
  pipeline.py    Anchor facade
benchmark/
  schema.py generator.py scenarios.py fillers.py
  systems/       baseline.py rag.py anchor.py counting_llm.py
  metrics.py runner.py report.py   cases/  results/
tests/  docs/ARCHITECTURE.md  AGENTS.md
docker-compose.yml  pyproject.toml
.github/workflows/ci.yml benchmark.yml
```

Environment	Setup
Local dev	pip install -e .; SQLite file; HashEmbedder and FakeLLM; pytest and ruff must pass
CI	GitHub Actions: ruff + pytest on every PR; benchmark with fake LLM, 30 cases, zero token cost
Production-like	docker compose: API container + Postgres with pgvector; keys through environment; real LLM through LLMClient
Release gates	Coverage at least 85%; backend contract suite green on both stores; tag v0.1.0 after integration

Team ownership and timeline

Person	Joins	Owns	Schedule
Person 1	hour 0	Memory side: core contracts, store, extraction, supersession, retrieval, ingest, API, Postgres	Hour 0-0.5 contract commit; 0.5-6 phases 1-4; 6-12 API, Postgres, CI; 12+ fixes from benchmark failures
Person 2	hour 0	Verification side: claim splitter, judge, decision engine, audit log, ask() and verify() wiring	Hour 0.5-6 phases 1-4 on FakeLLM; 6-12 service wiring and judge tuning; 12+ fixes
Person 3	hour 6	Benchmark: generator, scenarios, three systems, metrics, runner, CI workflow	Hour 6-12 phases 1-5; 12+ final test-split run and error analysis
Person 4	hour 12	Human-only work: API keys and budget, gold labels, related-work and novelty table, real-conversation tests, demo, README, deck, privacy review	Hour 12+ in parallel with final runs

Checkpoints: hour 0.5 contract commit on main. Hour 6 integration (Anchor.ingest and ask work end to end, v0.1.0). Hour 12 first full benchmark with failures.md. Working rules: one merged AGENTS.md, one phase per agent session, PRs from forks, only edit your own directories.

Roadmap and honest scope

v1 is this document. v2 candidates: graph layer for linked entities (Graphiti-style), embedding-based subject matching for supersession, tenant isolation, calibrated judge confidence. Before claiming novelty, compare against HaluMem, Memory-Observability, Graphiti, Engram and Mem0 (Person 4). The defensible claim is an evaluation-backed verification layer that sits on top of a memory store, not a new memory store.